Bluefish C:\Users\viby\ecen45632_f26\python\CourseDesign\Project1_explore\ClaudeAI_solution\student

```python
# from ClaudeAI solution
"""Student DSP module: STFT analysis/synthesis, FFT overlap-add filtering, jammer removal.

Conventions (match the assignment):
  X_m[k] = sum_r x[mH+r] w[r] exp(-j 2 pi k r / N),  periodic window, N >= L.
Real signals store the one-sided rFFT; complex (I/Q) signals store the full two-sided FFT.
"""
import numpy as np
from scipy import signal

FS = 16000


class NOLAError(ValueError):
    """Raised when sum_m w^2[n-mH] is (numerically) zero on a retained sample."""


# ---------------------------------------------------------------------------- STFT
def make_window(window, L):
    """Periodic window of length L (fftbins=True -> DFT-even, correct for STFT)."""
    if isinstance(window, np.ndarray):
        if len(window) != L:
            raise ValueError("window array length must equal L")
        return window.astype(float)
    name = {"rect": "boxcar", "rectangular": "boxcar"}.get(window, window)
    return signal.get_window(name, L, fftbins=True).astype(float)


def _layout(n, L, hop):
    """Padding: L-hop zeros in front so early samples get full overlap, and enough at the
    end that every retained sample is covered by the same number of frames."""
    pad_f = L - hop
    M = max(1, int(np.ceil((n + 2 * pad_f - L) / hop)) + 1)
    total = (M - 1) * hop + L
    return pad_f, total - n - pad_f, M


def analysis(x, L=1024, hop=None, N=None, window="hann", fs=FS):
    """Return (X, meta). X has shape (frames, N//2+1) for real x, (frames, N) for complex x."""
    x = np.asarray(x)
    cplx = np.iscomplexobj(x)
    x = x.astype(np.complex128 if cplx else np.float64)
    hop = L // 4 if hop is None else int(hop)
    N = L if N is None else int(N)
    if N < L:
        raise ValueError("N must be >= L")
    if not 1 <= hop <= L:
        raise ValueError("need 1 <= hop <= L")
    w = make_window(window, L)
    pad_f, pad_b, M = _layout(len(x), L, hop)
    xp = np.concatenate([np.zeros(pad_f, x.dtype), x, np.zeros(pad_b, x.dtype)])
    idx = hop * np.arange(M)[:, None] + np.arange(L)[None, :]
    frames = xp[idx] * w
    X = np.fft.fft(frames, n=N, axis=1) if cplx else np.fft.rfft(frames, n=N, axis=1)
    meta = dict(L=L, hop=hop, N=N, window=window, w=w, length=len(x), pad_front=pad_f,
                pad_back=pad_b, frames=M, complex=cplx, fs=fs)
    return X, meta


def frame_times(meta):
    """Frame-center times in seconds, with the leading padding offset removed."""
    return (meta["hop"] * np.arange(meta["frames"]) + meta["L"] / 2 - meta["pad_front"]) / meta["fs"]

```

```
63
64
65  def frame_freqs(meta):
66      """Bin frequencies in Hz: one-sided for real, two-sided (fftfreq order) for complex."""
67      d = 1 / meta["fs"]
68      return np.fft.fftfreq(meta["N"], d) if meta["complex"] else np.fft.rfftfreq(meta["N"],
    d)
69
70
71  def synthesis(X, meta, tol=1e-8):
72      """Weighted overlap-add with squared-window normalization.
73      Returns (y, denominator over retained samples). Raises NOLAError if NOLA fails."""
74      L, hop, N, w = meta["L"], meta["hop"], meta["N"], meta["w"]
75      M = X.shape[0]
76      v = np.fft.ifft(X, axis=1)[:, :L] if meta["complex"] else np.fft.irfft(X, n=N, axis=1)
    [:, :L]
77      total = (M - 1) * hop + L
78      idx = hop * np.arange(M)[:, None] + np.arange(L)[None, :]
79      num = np.zeros(total, v.dtype)
80      den = np.zeros(total)
81      np.add.at(num, idx, v * w)
82      np.add.at(den, idx, np.broadcast_to(w ** 2, idx.shape))
83      sl = slice(meta["pad_front"], meta["pad_front"] + meta["length"])
84      d = den[sl]
85      if d.min() <= tol * w.max() ** 2:
86          raise NOLAError(f"NOLA violated: min sum w^2 = {d.min():.3g} (window
    '{meta['window']}', "
87                          f"L={L}, hop={hop}); some samples are never (or barely) observed.")
88      return num[sl] / d, d
89
90
91  # ---------------------------------------------------------------------- FFT overlap-add
92  def ola_filter(x, h, B=512):
93      """Exact linear convolution by FFT overlap-add. Output length len(x)+len(h)-1."""
94      x = np.asarray(x)
95      h = np.asarray(h)
96      M, Nx = len(h), len(x)
97      nfft = 1 << int(np.ceil(np.log2(B + M - 1)))        # >= B+M-1 avoids circular aliasing
98      cplx = np.iscomplexobj(x) or np.iscomplexobj(h)
99      fft, ifft = (np.fft.fft, np.fft.ifft) if cplx else (np.fft.rfft, np.fft.irfft)
100     Hf = fft(h, nfft)
101     y = np.zeros(Nx + M - 1, np.complex128 if cplx else np.float64)
102     for s in range(0, Nx, B):                           # last block may be partial
103         blk = x[s:s + B]
104         yb = ifft(fft(blk, nfft) * Hf) if cplx else ifft(fft(blk, nfft) * Hf, nfft)
105         n = len(blk) + M - 1                            # full block convolution length
106         y[s:s + n] += yb[:n]
107     return y
108
109
110 # ---------------------------------------------------------------------- jammer removal
111 def _odd(n):
112     return n if n % 2 else n - 1
113
114
115 def estimate_ridge(X, meta, band=(1600, 3400), fs=FS):
116     """Dominant frequency per frame inside `band` (parabolic interpolation on log-mag),
117     then median + Savitzky-Golay smoothing. Uses only the mixture STFT."""
118     f = frame_freqs(meta)
119     sel = np.where((f >= band[0]) & (f <= band[1]))[0]
120     mag = np.abs(X[:, sel])
121     k = np.argmax(mag, axis=1)
122     peak = mag[np.arange(len(k)), k]
123     raw = f[sel][k].copy()
124     km = np.clip(k, 1, len(sel) - 2)
125
```

```
125     a, b, c = (np.log(mag[np.arange(len(k)), km + d] + 1e-30) for d in (-1, 0, 1))
126     den = a - 2 * b + c
127     off = np.where(np.abs(den) > 1e-12, 0.5 * (a - c) / den, 0.0)
128     raw = f[sel][km] + np.clip(off, -1, 1) * (f[1] - f[0])
129     # frames with no real energy in band (pure padding) carry no information: interpolate
130     valid = peak > 0.02 * np.median(peak[peak > 0]) if np.any(peak > 0) else
    np.zeros_like(peak, bool)
131     m = np.arange(len(raw))
132     if valid.sum() >= 2:
133         raw = np.interp(m, m[valid], raw[valid])
134     M = len(raw)
135     r = signal.medfilt(raw, _odd(min(7, M)) if M >= 3 else 1)
136     wl = _odd(min(21, M))
137     if wl >= 5:
138         r = signal.savgol_filter(r, wl, 2, mode="interp")
139     return r, raw, peak
140
141
142 def jammer_mask(ridge_hz, freqs, width_hz=60.0, gmin=0.03):
143     """G[m,k] = 1-(1-gmin) exp(-(|f_k|-f_m)^2/(2 sigma^2)); symmetric in +/-f by using |
    f|."""
144     d = np.abs(freqs)[None, :] - np.asarray(ridge_hz)[:, None]
145     return 1 - (1 - gmin) * np.exp(-d ** 2 / (2 * width_hz ** 2))
146
147
148 def apply_mask(x, G, meta_like):
149     """Apply a frozen mask G (frames x bins) to any equal-length real signal with the same
    framing."""
150     X, meta = analysis(x, L=meta_like["L"], hop=meta_like["hop"], N=meta_like["N"],
151                        window=meta_like["window"], fs=meta_like["fs"])
152     return synthesis(X * G, meta)[0]
153
154
155 def remove_jammer(x, fs=FS, band=(1600, 3400), L=1024, hop=None, width_hz=60.0,
156                   gmin=0.03, window="hann", stationary_hz=None):
157     """Remove a moving narrowband jammer from the mixture only (no clean/jammer reference).
158     stationary_hz: if given, use one fixed notch there instead of the tracked ridge
    (control).
159     Returns (y, diagnostics). diagnostics['G'] and ['meta'] allow freezing the mask."""
160     X, meta = analysis(x, L=L, hop=hop, window=window, fs=fs)
161     f = frame_freqs(meta)
162     ridge, raw, peak = estimate_ridge(X, meta, band, fs)
163     track = np.full_like(ridge, stationary_hz) if stationary_hz is not None else ridge
164     G = jammer_mask(track, f, width_hz, gmin)
165     y, _ = synthesis(X * G, meta)                          # complex phases untouched
166     diag = dict(ridge_hz=ridge, raw_ridge_hz=raw, peak=peak, track_hz=track, G=G, meta=meta,
167                 times=frame_times(meta), freqs=f, width_hz=width_hz, gmin=gmin, band=band)
168     return y, diag
169
```